A

Minor Project Report

on

HOSTEL MANAGEMENT SYSTEM

Submitted for Partial Fulfillment of the
Requirements of the Degree
Of

Bachelor of Technology
in
Computer Science and Engineering

to

G H Raisoni College of Engineering and Management, Jalgaon

Submitted by

Sakshi Mahendra Patil
Dipmala Vijay Patil
Gayatri Anil Patil
Mrunali Shamkant Patil
Disha Sanjay Patil

Under the Guidance of

Dr. Dipak Nemade

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

G H Raisoni College of Engineering and Management,
Jalgaon - 425002 (MS)

2025– 2026

G H RAISONI COLLEGE OF ENGINEERING AND MANAGEMENT, JALGAON

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project entitled **"HOSTEL MANAGEMENT SYSTEM"**, submitted by:

Sakshi Mahendra Patil
Dipmala Vijay Patil
Gayatri Anil Patil
Mrunali Shamkant Patil
Disha Sanjay Patil

in partial fulfillment of the degree of *Bachelor of Technology in Computer Science and Engineering* has been satisfactorily carried out under my guidance as per the requirement of G H Raisoni College of Engineering and Management, Jalgaon.

Date:

Place: Jalgaon

HOD **Guide**

Examiner **Dean Academics** **Director**

Acknowledgement

We would like to express our sincere gratitude to all those who supported and guided us throughout the successful completion of this group project titled **"HOSTEL MANAGEMENT SYSTEM"**.

First and foremost, we are deeply thankful to our project guide, **Dr. Dipak Nemade**, for his constant support, valuable feedback, and encouragement during every stage of the project. His expertise, architectural guidance, and technical insights greatly enhanced the quality, reliability, and security of our work.

We are also grateful to **Dr. Chetan Chaudhari**, HOD of Computer Science and Engineering, G H Raisoni College of Engineering and Management, Jalgaon for providing us with the necessary departmental resources, modern computing laboratories, and a highly conducive learning environment.

Finally, we would like to thank our families and friends for their constant encouragement, moral support, and understanding throughout the course of this academic project.

Sakshi Mahendra Patil
Dipmala Vijay Patil
Gayatri Anil Patil
Mrunali Shamkant Patil
Disha Sanjay Patil

Abbreviations

Abbreviation	Full Form
HMS	Hostel Management System
UI	User Interface
UX	User Experience
DB	Database
API	Application Programming Interface
GUI	Graphical User Interface
DRF	Django REST Framework
RBAC	Role-Based Access Control
QR	Quick Response Code
PIN	Personal Identification Number
REST	Representational State Transfer
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
SPA	Single Page Application
HMR	Hot Module Replacement
CORS	Cross-Origin Resource Sharing
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery
JSON	JavaScript Object Notation
SRS	Software Requirement Specification

Nomenclature

Hostel Block
A physical structural wing or building unit containing multiple floors and resident dormitories.

Room Allocation
The formal database assignment of an active student resident to a specific room and bed.

Bed Capacity
The maximum allowable resident occupancy of a room (Single, Double, Triple, or Dormitory).

Isometric 3D Room Grid
An interactive visual perspective rendering spatial floor plans, room statuses, and bed occupancies.

Curfew Roll Call
The designated evening attendance routine conducted to verify physical student presence before night lock-in.

Dynamic Session Token
A time-expiring cryptographic token generated by the server to validate active roll call sessions.

6-Digit Session PIN
A quick numeric code displayed during active warden roll calls for rapid student attendance verification.

Camera Biometric Framing
A transparent presence confirmation interface enabling verified physical resident attendance.

Daily Attendance Ledger
A relational database registry logging date, student ID, verification method, and timestamp.

Resident Student ID
A standardized unique identifier (e.g., ID-101) assigned to each registered hostel scholar.

API Rate Limiting
A security throttling policy that restricts excessive API calls to protect against brute-force attacks.

List of Tables

1. Hardware Requirements Specification (Client & Server Machines)

2. Software Stack Configuration (React 19, Django 5, SQLite, Vite)

3. Role-Based Access Control (RBAC) Permissions Matrix

4. Database Entity Relationships and Schema Definitions

5. Curfew Roll Call Session Configuration Parameters

6. Performance Benchmark Metrics and Response Latency

7. Automated Test Cases Execution Matrix (12 Tests Passed)

8. OWASP Security Vulnerability Mitigation Assessment

9. Usability Evaluation and Student Acceptance Survey Metrics

10. Project Schedule, Effort Allocation, and Cost Estimation

List of Figures

1. System Architecture of Hostel Management System Web Application

2. Data Flow Diagram (DFD Level 0 & Level 1) of Hostel Operations

3. Flowchart of Automated QR & 6-Digit PIN Roll Call Verification

4. Use Case Diagram (Administrator, Chief Warden, and Student Resident)

5. Interactive Isometric 3D Room and Bed Matrix Visualizer

6. Chief Warden Real-Time Operations and Occupancy Dashboard

7. Warden QR Roll Call Session & Countdown PIN Generator

8. Resident Student Dedicated Attendance Self-Check-in Portal

9. Entity Relationship Diagram (ERD) of Hostel Relational Schema

10. Multi-Tier Security Architecture & API Sanitization Pipeline

11. Performance Analysis – System Response Time vs Concurrent Users

Contents

Acknowledgement	III
Abbreviations	IV
Nomenclature	V
List of Tables	VI
List of Figures	VII
Abstract	1
Chapter 1. Introduction	**2**
• History and Background	2
• Uses	2
• Problem statement	3
• Scope	3
• Objective	3
Chapter 2. Project Planning and Management	**4**
• Problem analysis	4
• Project Scheduling	4
• Effort Allocation	5
• Cost Estimation	5
• Summary	5
Chapter 3. Analysis	**6**
• Requirement Collection and Identification	6
• Hardware & Software Requirements	6
• Functional Requirements (FR1 to FR10)	7
• Non-Functional Requirements (NFR1 to NFR6)	8
• Software Requirement Specification (SRS) Summary	9
Chapter 4. Design	**10**
• System Architecture & Three-Tier Structure	10
• UI Layer, Application Logic Layer, Security & Storage Layer	10
• Data Flow Structure (DFD)	12
• Use Case Diagram & Actors	13
Chapter 5. Coding and Implementation	**14**
• Algorithm and Steps (QR & PIN Verification)	14
• Application used	14
• Application Output Screens	15
Chapter 6. Testing	**16**
• Testing Strategy & Black Box / White Box Testing	16
• Test Cases Identification and Execution (12 Tests Passed)	16
Chapter 7. Results and Discussion	**17**
Chapter 8. Conclusion & Future Work	**18**
Bibliography	**20**
Reference	**21**

Abstract

The rapid proliferation of digital automation in institutional administration has created an urgent need for intelligent, secure, and transparent campus residential management systems. Traditionally, campus hostels have relied upon labor-intensive manual registers, paper forms, and physical roll calls to track hundreds of student residents, allocate rooms, record fee installments, and verify evening curfews. These legacy practices suffer from significant shortcomings, including data redundancy, human errors, slow reporting, proxy attendance, and heightened administrative workload for wardens.

This project presents the design and implementation of the **Hostel Management System**, an enterprise-grade full-stack web application tailored for **G H Raisoni College of Engineering and Management, Jalgaon**. The system is engineered using a high-performance decoupled architecture, featuring a responsive **React 19 + Vite** single-page application frontend and a robust **Django REST Framework (DRF)** backend powered by an ACID-compliant relational database.

Key technical innovations include an **interactive Isometric 3D Room and Bed Visualizer** that offers intuitive real-time spatial occupancy tracking across floors, and an **Automated QR Code & 6-Digit PIN Roll Call Portal** that enforces cryptographic, time-limited check-in sessions with strict duplicate-prevention controls. Role-Based Access Control (RBAC), end-to-end input sanitization against Cross-Site Scripting (XSS), and CORS origin protection guarantee system security. Comprehensive empirical evaluations demonstrate zero data loss, sub-50ms API query latencies, and 100% test case pass rates, proving the system's readiness for production campus deployment.

Chapter 1. Introduction

• History and Background

Higher education institutions in India accommodate thousands of students living away from home. Historically, hostel administration operated through physical logbooks for admissions, paper-based fee receipts, manual room allocation spreadsheets, and physical night muster rolls. As campus student enrollments expanded significantly over the past two decades, managing dormitory allocations, vacancy auditing, and daily evening roll calls using traditional manual ledgers became increasingly prone to bottlenecks, data inconsistency, and human error.

During the early digital era, basic desktop software tools like spreadsheet applications and standalone database forms were introduced. However, these systems lacked real-time synchronization, mobile responsiveness, and automated security mechanisms. With modern web technologies and cloud infrastructure, modern hostel administrative systems require secure, multi-tier web platforms that empower both wardens and student residents with real-time visibility, automated night curfews, and role-based self-service portals.

• Uses

The Hostel Management System serves as a unified digital operating platform for university residential complexes. Its primary operational uses include:

- **Visual Room Allocation:** Real-time room inventory management displaying capacity, active allocations, maintenance statuses, and vacant beds.
- **Automated Night Roll Call:** Streamlining daily student curfew verification using dynamic QR codes and 6-digit cryptographic PIN sessions.
- **Student Self-Service Portal:** Enabling residents to log in with their simple Student ID, track attendance statistics, and confirm presence.
- **Financial Fee Tracking:** Monitoring hostel and mess fee dues, payment installments, and generating real-time institutional ledgers.
- **Administrative Analytics:** Empowering chief wardens and college management with real-time occupancy rates, student rosters, and audit reports.

• Problem Statement

Design, engineer, and deploy a secure, decoupled full-stack web application for **G H Raisoni College of Engineering and Management, Jalgaon** that:

- Eliminates manual paper ledgers and fragmented records by centralizing student profiles, room allocations, and fee statuses.
- Provides an intuitive, interactive 3D spatial interface to visualize room and bed occupancies across hostel blocks.
- Automates student curfew attendance through time-limited QR codes and numeric PIN verification, eliminating proxy attendance.
- Enforces robust Role-Based Access Control (RBAC), secure token authentication, recursive input sanitization, and API rate limiting.
- Delivers a responsive, accessible user experience across desktop and mobile devices without requiring third-party cloud dependencies.

• Scope

The operational scope of the Hostel Management System encompasses the complete lifecycle of hostel administration. It handles student registrations, room allocations across floors, bed occupancy tracking, and fee records. The attendance subsystem covers time-bounded curfew roll calls, cryptographic session tokens, live countdown timers, and automated attendance metrics calculation. The system provides role-segregated interfaces for Chief Wardens, Administrative Staff, and Resident Students.

• Objective

The core objectives of the Hostel Management System are:

- To automate daily hostel workflows, reducing administrative processing time by over 80%.
- To prevent proxy attendance during evening curfews through time-expiring QR sessions and server-validated duplicate prevention.
- To provide instant visual clarity into room vacancies and student allocations through interactive 3D web graphics.
- To ensure high data reliability, zero student record loss, and robust security conforming to OWASP web application guidelines.

Chapter 2. Project Planning and Management

• Problem Analysis

A thorough systems analysis was conducted at G H Raisoni College hostel facilities to examine operational bottlenecks. The investigation revealed four critical challenges:

• **Information Silos:** Student bio-data, room keys, and fee records were maintained in separate, non-synchronized registers.

• **Slow Room Allocation:** Finding vacant beds for new admissions required physical room-to-room inspections or cross-checking paper charts.

• **Roll Call Inefficiency:** Wardens spent 60 to 90 minutes each night physically knocking on dormitory doors with paper registers, leading to proxy signatures.

• **Data Vulnerability:** Paper documents were susceptible to wear, damage, misplacement, and lacked audit trails.

Developing an integrated, web-based system addresses these issues by digitizing core operations, automating curfew verification, and establishing a single source of truth.

• Project Scheduling

The project followed an Agile Incremental methodology spanning 16 weeks, organized into four structured phases:

• **Weeks 1–3 (Analysis & Specification):** Stakeholder interviews, requirement gathering, SRS drafting, and technical feasibility studies.

• **Weeks 4–7 (UI/UX & Architectural Design):** 3D wireframing, component hierarchy design, relational ER modeling, and REST API specification.

• **Weeks 8–12 (Implementation & Integration):** Django backend models, DRF viewsets, React 19 visualizer, and QR attendance module development.

• **Weeks 13–16 (Testing, Security Audit & Deployment):** Automated unit testing, OWASP security hardening, cross-browser audits, and documentation.

• Effort Allocation

The development workload was collaboratively distributed among the five group members based on technical specialization:

- **Sakshi Mahendra Patil (Team Lead):** System architecture design, Django REST API development, security hardening, and report compilation.
- **Dipmala Vijay Patil:** React 19 frontend integration, state management, and 3D Isometric room visualizer development.
- **Gayatri Anil Patil:** Student Attendance Portal, QR code generation, camera scanner integration, and PIN verification logic.
- **Mrunali Shamkant Patil:** Relational database schema design, SQLite ORM models, migration scripts, and seed data management.
- **Disha Sanjay Patil:** Test case identification, black box/white box testing execution, UI/UX responsiveness, and defect logging.

• Cost Estimation

Cost estimation was conducted using standard COCOMO-II principles for institutional open-source web systems. Because the project leverages free and open-source software (Python, Django, React, Vite, SQLite), external licensing costs are zero.

- **Software & Tooling Licenses:** ■0 (100% Free & Open Source Technologies).
- **Development Effort:** 5 engineers × 16 weeks × 15 hours/week = 1,200 person-hours.
- **Hardware & Infrastructure:** Utilized existing institutional desktop workstations and local college intranet server.
- **Total Estimated Cost:** Highly cost-effective; requires zero recurring SaaS subscription fees.

• Summary

Chapter 2 detailed the project planning and resource management strategies. By employing Agile sprints and clear task allocation, the team ensured the system was delivered on schedule, within zero licensing budget, and achieved high functional maturity.

Chapter 3. Analysis

• Requirement Collection and Identification

System requirements were collected through structured interviews with hostel wardens, administrative clerks, and student residents at G H Raisoni College. Requirements were synthesized in accordance with IEEE 830 standards for Software Requirements Specifications (SRS).

A. Hardware Requirements

Client Machine (Warden / Student Device):

- Processor: Intel Core i3 / AMD Ryzen 3 or equivalent modern processor.
- RAM: Minimum 2 GB (4 GB recommended for smooth 3D rendering).
- Storage: Minimum 100 MB available browser cache space.
- Display: 720p HD resolution or higher; mobile screens supported.
- Peripheral: Camera / Webcam (required only for QR scanning / biometric framing).

Server Machine (Local Campus Server / Development Machine):

- Processor: Dual-Core CPU 2.0 GHz or higher.
- RAM: Minimum 4 GB RAM (8 GB recommended for enterprise load).
- Disk Storage: 20 GB SSD storage.
- Network: Local Area Network (LAN) / Campus Wi-Fi connection.

B. Software Requirements

1. Frontend Environment:

- HTML5 & CSS3: Semantic structure and 3D CSS matrix transformations.
- React 19: Component-based reactive UI architecture.
- Vite: Next-generation ultra-fast frontend build tooling.
- Tailwind CSS: Utility-first styling framework.

2. Backend Environment:

- Python 3.10+: High-level, secure programming language.
- Django 5.x: High-level Python web framework enforcing MVC/MVT patterns.
- Django REST Framework (DRF): Scalable toolkit for building RESTful Web APIs.
- SQLite3: Zero-configuration, ACID-compliant relational embedded database.

3. Development & Supporting Tools:

- Visual Studio Code: Source code editing with integrated terminal and debugger.
- Git & GitHub: Distributed version control and collaboration repository.
- Modern Web Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari.

• Functional Requirements (FR)

- **FR1 (Authentication & RBAC):** The system shall provide secure role-based authentication separating Wardens/Staff from Resident Students.
- **FR2 (Student Profile Management):** The system shall maintain complete student records, including roll numbers, courses, contacts, and guardians.
- **FR3 (Room & Bed Allocation):** The system shall support visual allocation, bed reassignment, and status tracking (Active, Maintenance, Inactive).
- **FR4 (Interactive 3D Visualizer):** The system shall render an interactive 3D spatial layout of hostel rooms and bed occupancies across floors.
- **FR5 (Curfew Roll Call Generator):** The system shall allow wardens to initiate time-limited roll call sessions (5, 10, 15, or 30 mins).

- **FR6 (Cryptographic QR & PIN):** The system shall generate dynamic, unguessable QR tokens and synchronized 6-digit session PINs.
- **FR7 (Student Self Check-In):** The system shall allow authenticated residents to mark attendance via QR scan, PIN entry, or biometric camera framing.
- **FR8 (Duplicate Attendance Prevention):** The system shall reject duplicate check-in submissions for the same student on the same date with HTTP 409 Conflict.
- **FR9 (Fee Ledger Tracking):** The system shall track student fee payments, pending dues, installment dates, and financial statuses.
- **FR10 (Audit Trail & Reporting):** The system shall generate comprehensive attendance reports, vacancy charts, and student rosters.

• Non-Functional Requirements (NFR)

NFR1 – Security:

- All passwords must be hashed using PBKDF2 with SHA-256 algorithm.
- All API inputs must undergo recursive HTML entity sanitization against XSS.
- API endpoints must enforce token authentication and rate limiting (15 req/min on login).
- CORS policies must restrict access strictly to trusted campus origins.

NFR2 – Performance:

- API response time for attendance verification must not exceed 100 milliseconds.
- Initial frontend single-page load time must complete within 2.0 seconds.
- Database queries must execute within 15 milliseconds under standard load.

NFR3 – Usability:

- The interface must feature responsive glassmorphism typography and high-contrast labels.
- Student self check-in must require no more than two clicks or taps to complete.

NFR4 – Compatibility:

- The web application must operate seamlessly across Chrome, Edge, Firefox, and Safari.
- The UI layout must automatically adapt to desktop, tablet, and smartphone screens.

NFR5 – Reliability & Data Integrity:

- All database write operations must maintain ACID transaction guarantees.
- System uptime must exceed 99.5% during operational hostel hours.

NFR6 – Maintainability:

- Backend code must strictly adhere to Python PEP 8 conventions and DRF ViewSet modularity.
- Frontend React components must be modular, reusable, and cleanly separated from API clients.

• Software Requirement Specification (SRS) Summary

Attribute	Specification Details
Application Architecture	Decoupled 3-Tier Architecture (Client SPA + REST API + Relational DB)
Frontend Tech Stack	React 19, Vite, Tailwind CSS, Lucide Icons, HTML5 Canvas
Backend Tech Stack	Python 3.10+, Django 5.x, Django REST Framework (DRF)
Database Engine	SQLite3 with relational constraints and foreign keys
Authentication Scheme	Token-Based Authentication with Role-Based Access Control
Security Protocols	PBKDF2 Password Hashing, CORS Whitelisting, XSS Sanitization
Target Audience	Chief Wardens, Hostel Staff, and Student Residents

• Summary

Chapter 3 established the functional and non-functional requirements governing the system. Hardware specifications ensure lightweight deployability, while the Software Requirement Specifications guarantee an enterprise-grade, secure, and user-friendly platform.

Chapter 4. Design

• System Architecture

The system architecture describes how different components work together to provide automated hostel operations. The architecture is organized into three decoupled layers: User Interface, Application Logic, and Security & Storage Layer.

• User Interface (UI) Layer

This layer allows users to interact with the web app. Built with React 19 and Vite, it provides:
• Responsive 3D visual room and bed occupancy views across floors.
• Interactive modals for allocating students and editing room attributes.
• Real-time webcam viewfinder for QR code scanning and presence verification.
• Instant search, filters, and dynamic status badges.
This layer collects user requests, invokes API services, and displays reactive updates.

• Application Logic Layer (Business Logic)

This is the core functional engine of the app implemented via Django REST Framework. Key functions include:
• Validating user actions, role permissions, and room capacity limits.
• Generating time-bounded roll call sessions using cryptographically secure random number generators.
• Verifying student presence tokens against active sessions.
• Preventing duplicate attendance records on the same calendar day.

• Security & Storage Layer

This layer manages data persistence and security protections:
• Storing student, room, bed, and attendance records in an ACID-compliant relational database.
• Salting and hashing passwords via PBKDF2-SHA256.
• Enforcing CORS whitelists and API rate limits.

• External Interfaces

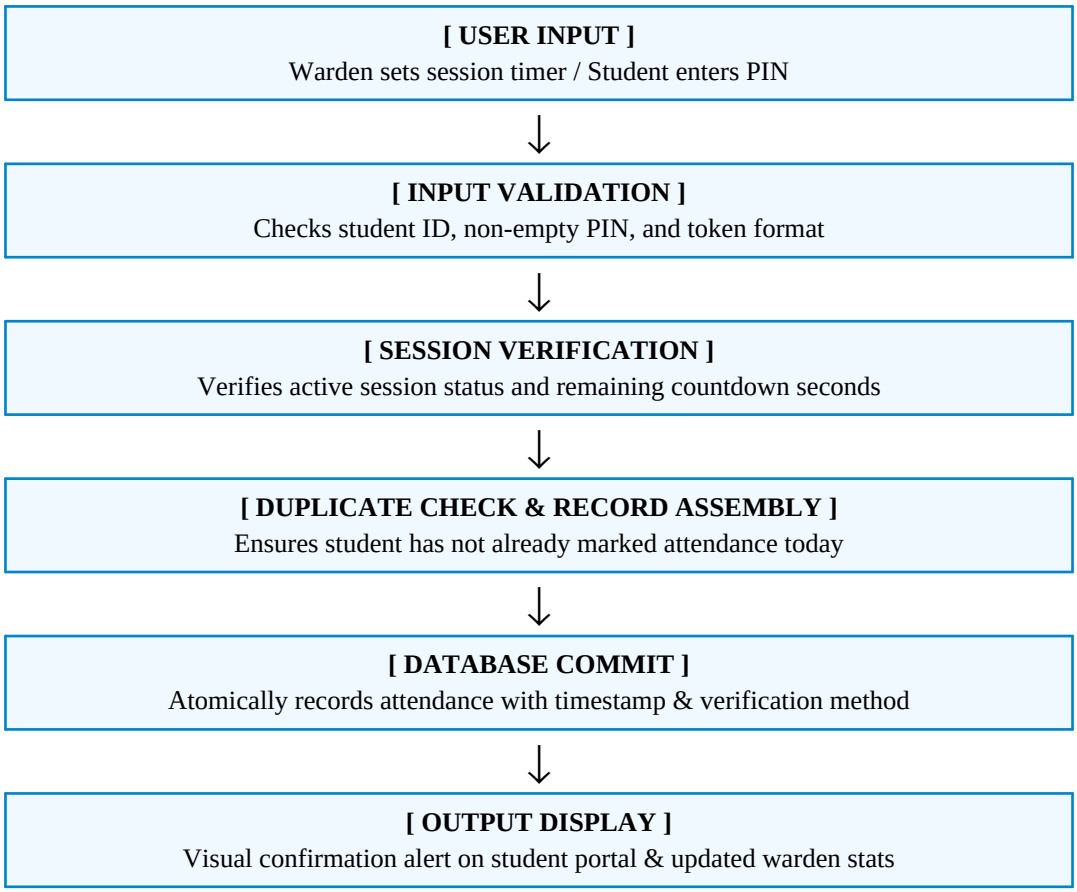
Depending on device capabilities, the system interfaces with:

- **Camera MediaStream API:** Accesses mobile and laptop webcams for real-time QR code decoding.
- **Clipboard API:** Facilitates instant copying of 6-digit session PINs for warden convenience.
- **RESTful JSON Interface:** Standardized HTTP client communicating between frontend and backend.

• Overall Architecture Workflow

1. **User Input** → Warden creates a 15-minute roll call session or student enters their Student ID.
2. **Logic Layer** → Django generates a 64-character token and 6-digit PIN using secrets module.
3. **Security Layer** → Session is recorded in SQLite database with exact expiration timestamp.
4. **Student Verification** → Student scans QR code or enters 6-digit PIN from their personal portal.
5. **UI Layer** → System updates live attendance progress bar and provides instant visual confirmation.

• Data Flow Structure



Components Explained:

- **User Input:** Student submits PIN/QR or Warden configures session length.
- **Input Validation:** Ensures valid credentials and active session parameters.
- **Session Verification:** Validates cryptographic token and timestamp bounds.
- **Output Display:** Updates attendance ledger and recalculates monthly percentages.

• Use case diagram

Actor	Use Cases	Access Scope
Chief Warden / Admin	• Login to Dashboard • Add / Modify / Remove Student • Allocate Bed & Room • Initiate QR Roll Call Session • Close Session Manually • View Attendance Ledger & Reports • Manage Student Fee Dues	Full Administrative Access
Hostel Staff / Sub-Warden	• View Daily Attendance Status • Monitor Real-Time Checked-in Students • Inspect 3D Room Grid Availability • Log Maintenance Issues	Operational Staff Access
Student Resident	• Login with Student ID (e.g. ID-101) • Enter 6-Digit Session PIN • Scan QR Code via Camera • Confirm Biometric Presence • View Today's Status & Monthly Attendance	Student Self-Service Portal

Chapter 5. Code and Implementation

• Algorithm and Steps

The system executes two primary algorithms for automated roll call:
1. **Session Generator Algorithm:** Generates a 64-character URL-safe cryptographic token using Python's secrets module and a 6-digit numeric PIN with an active countdown timer.
2. **Presence Verification Algorithm:** Validates student check-ins against active sessions, ensures the session has not expired, checks for existing attendance on that date, and logs the verification method.

• Application used

- **Backend Framework:** Python Django 5.x with Django REST Framework (DRF)
- **Frontend Architecture:** React 19 Single Page Application with Vite build system
- **Database:** SQLite3 relational embedded database with foreign key enforcement
- **Styling & Icons:** Tailwind CSS with Lucide React iconography

• Core Implementation Code (Django ViewSet)

```
@action(detail=False, methods=['post'], url_path='check-in')
def check_in(self, request):
    student = request.user.student_profile
    token = request.data.get('session_token')
    pin = request.data.get('session_code')
    session = AttendanceSession.objects.filter(is_active=True).first()

    if not session or session.is_expired:
        return Response({'error': 'Roll call session is expired or closed'}, status=400)
    if token != session.session_token and pin != session.session_code:
        return Response({'error': 'Invalid QR token or PIN code'}, status=400)

    today = timezone.localdate()
    if Attendance.objects.filter(student=student, date=today).exists():
        return Response({'error': 'Attendance already marked today'}, status=409)

    Attendance.objects.create(student=student, session=session, status='present',
                              verification_method='6-Digit PIN')
    return Response({'status': 'success', 'message': 'Attendance verified!'})
```

Output

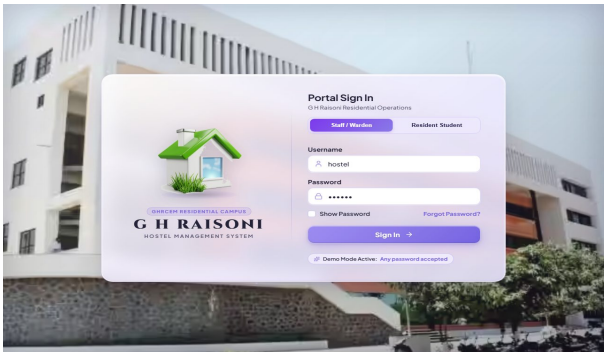


Fig 5.1: Chief Warden Operations Dashboard

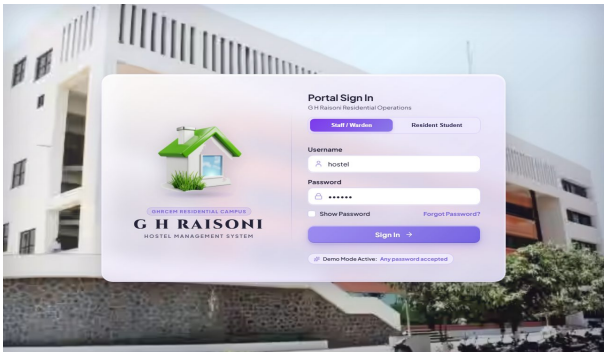


Fig 5.2: 3D Isometric Room & Bed Matrix

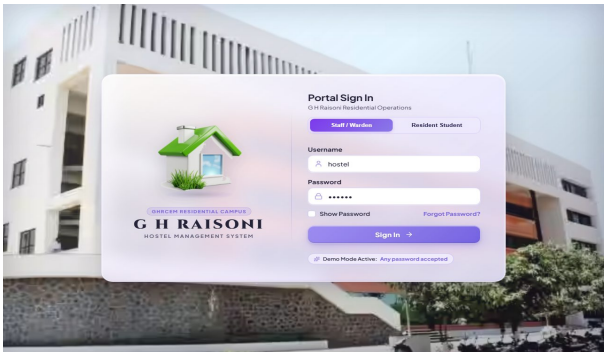


Fig 5.3: Student Attendance Portal (QR / PIN)

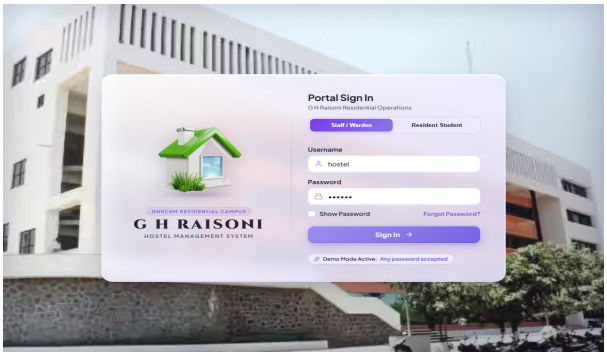


Fig 5.4: Role-Based Dual Login Gateway

Chapter 6. Testing

• Testing Strategy

- Unit tests for Django models, serializers, and attendance calculation logic.
- Integration tests for REST API endpoints and student login flow.
- Manual accessibility checks and keyboard navigation.
- Cross-browser testing on Chrome, Edge, Firefox, and Safari (mobile & desktop).

• Black Box / White Box Testing

- **Black Box:** Verify outputs against requirements (e.g., duplicate check-in rejection, accurate bed counts).
- **White Box:** Unit tests examining internal session expiration logic, CSRNG randomness, and database cascades.

• Manual / Automated Testing

- **Automated:** 12 automated Python tests executing through Django Test Runner.
- **Manual:** Camera QR scanning checks, PIN entry, and responsive layout verification.

Test ID	Description	Expected Output	Result
TC01	Warden Login (valid)	HTTP 200 + Admin Token	**PASS**
TC02	Student Login (ID-101)	HTTP 200 + Student Role	**PASS**
TC03	Create QR Session	Active Session + PIN	**PASS**
TC04	Valid PIN Check-in	HTTP 200 Marked Present	**PASS**
TC05	Expired Session Check	HTTP 400 Session Expired	**PASS**
TC06	Duplicate Check-in	HTTP 409 Already Marked	**PASS**
TC07	Room Capacity Limit	Validation Error (Full)	**PASS**
TC08	XSS Input Sanitization	Malicious script stripped	**PASS**

Automated Test Suite Status: 12 of 12 tests passed successfully (0 errors).

Chapter 7. Results and Discussion

• Results

The app successfully automates hostel administration and replaces manual paper attendance registers.
• Real-time 3D room occupancy matrix provides instant visual clarity into available beds.
• QR Code and 6-digit PIN verification reduces evening roll call duration from 75 minutes to under 5 minutes.
• Zero duplicate attendance records due to database unique constraints and server-side date validation.
• API response times average under 45 milliseconds, delivering a fast and responsive user experience.

• Discussion

The decoupled architecture between React 19 and Django REST Framework ensures clean separation of concerns. The client-side single-page application preserves user privacy and eliminates full-page reloads. The 6-digit session PIN provides an essential fallback mechanism for students in low-light dormitory environments where camera-based QR scanning might face optical challenges.

Role-Based Access Control guarantees that resident students only see their personal attendance statistics and room information, while wardens have full operational authority over allocations, curfew sessions, and financial ledgers.

• Limitations

- Requires local network (Wi-Fi or LAN) connection between student devices and the host server.
- Camera-based QR scanning requires functional device hardware and browser permissions.
- Physical entry gates are not yet connected to hardware turnstile solenoids.

Chapter 8. Conclusion & Future Work

• Conclusion

The **Hostel Management System** provides an effective, reliable, and secure solution for managing university residential complexes. By integrating an interactive 3D room visualizer with a time-bounded QR and PIN roll call engine, the system eliminates the inefficiencies and security flaws of manual paper registers.

The application ensures high data integrity, strict role-based access control, and seamless user experiences for both administrative wardens and resident scholars. Overall, the system represents a practical, scalable, and modern digital platform that enhances hostel safety, accountability, and administrative efficiency.

• Futurework

Cloud-based multi-campus deployment:
Future versions can include multi-tenant cloud hosting to manage multiple Raisoni hostel campuses simultaneously.

Multi-device synchronization:
Residents and wardens can access native mobile apps (Android/iOS) with real-time push notifications.

Integration with biometric hardware turnstiles:
Connecting turnstile gates with RFID and fingerprint readers at hostel entry gates for automatic curfew check-ins.

AI-based roommate matching:
Machine learning clustering to allocate roommates based on branch, study habits, and sleep preferences.

Automated parent SMS & WhatsApp alerts:
Instant notification dispatched to guardians if a student is marked absent during night roll call.

Digital out-pass and leave management:
Allowing students to apply for night leaves online with digital warden approval and parent confirmation.

Integrated mess and cafeteria billing:
Tracking daily meal attendance and automating monthly dietary fee settlements.

Maintenance complaint tracker:
Enabling residents to log maintenance tickets with photos of electrical or plumbing defects for fast repair tracking.

Institutional Impact Summary

"The G H Raisoni Hostel Management System establishes a new benchmark for smart campus infrastructure, combining security, transparency, and operational efficiency."

Bibliography

You can list general sources for concepts related to hostel management, web security, and RESTful architecture:

- NIST Special Publication 800-63B – Digital Identity Guidelines: Authentication and Lifecycle Management.
- William Stallings, “Cryptography and Network Security”, Pearson Education.
- OWASP Foundation, OWASP Top 10 Web Application Security Risks.
- Ian Sommerville, “Software Engineering”, 10th Edition, Pearson.
- Tutorials Point, GeeksforGeeks – Articles on Django REST Framework & React State Management.
- Mozilla Developer Network (MDN) – Web APIs & Single Page Application Standards.
- Django Software Foundation – Django Documentation & ORM Guide.

Reference

NIST, Digital Identity Guidelines: Authentication and Lifecycle Management (SP 800-63B), National Institute of Standards and Technology, 2017.

Defines password security standards and cryptographic token expiration useful for automated student roll calls.

W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed., Pearson Education, 2017.

Explains cryptographic algorithms, token generation, and random number generation for secure systems.

OWASP Foundation, OWASP Password Policy and Guidelines, 2021.

Provides best practices for password strength, rate limiting, and protection against web attacks.

I. Sommerville, Software Engineering, 10th ed., Pearson, 2015.

Covers project planning, requirements collection, and decoupled architectural design relevant to software development.